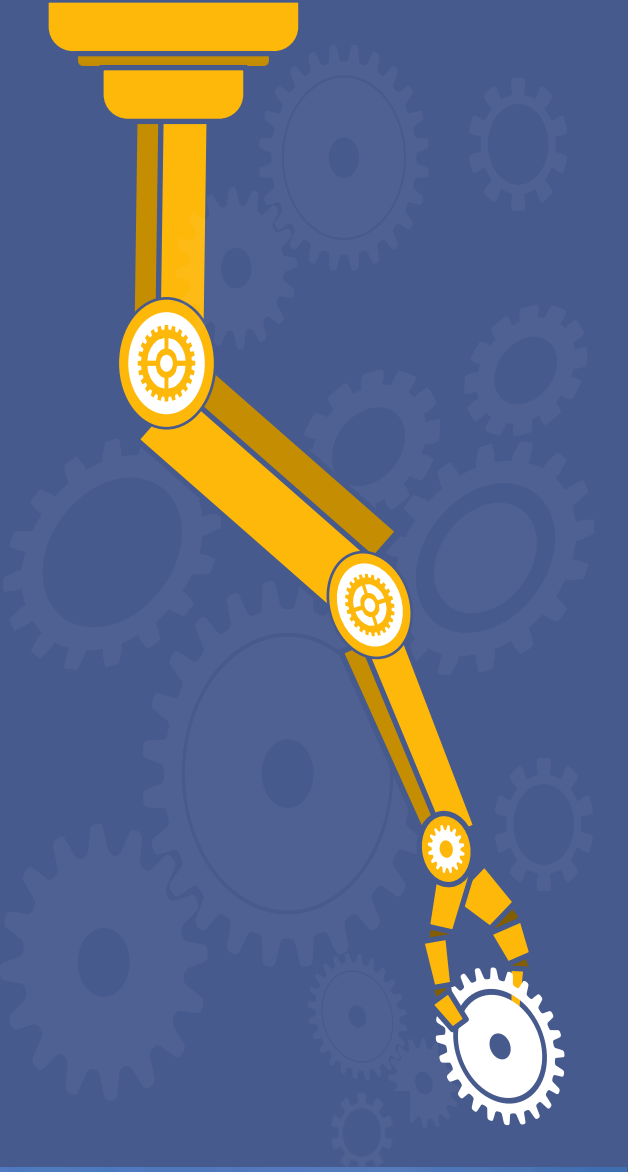


VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT, Vellore.

BCSE204L- Design and Analysis of Algorithms-

Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms

2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively

3. To synthesize efficient algorithms in various engineering design situations

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms- Course outcome

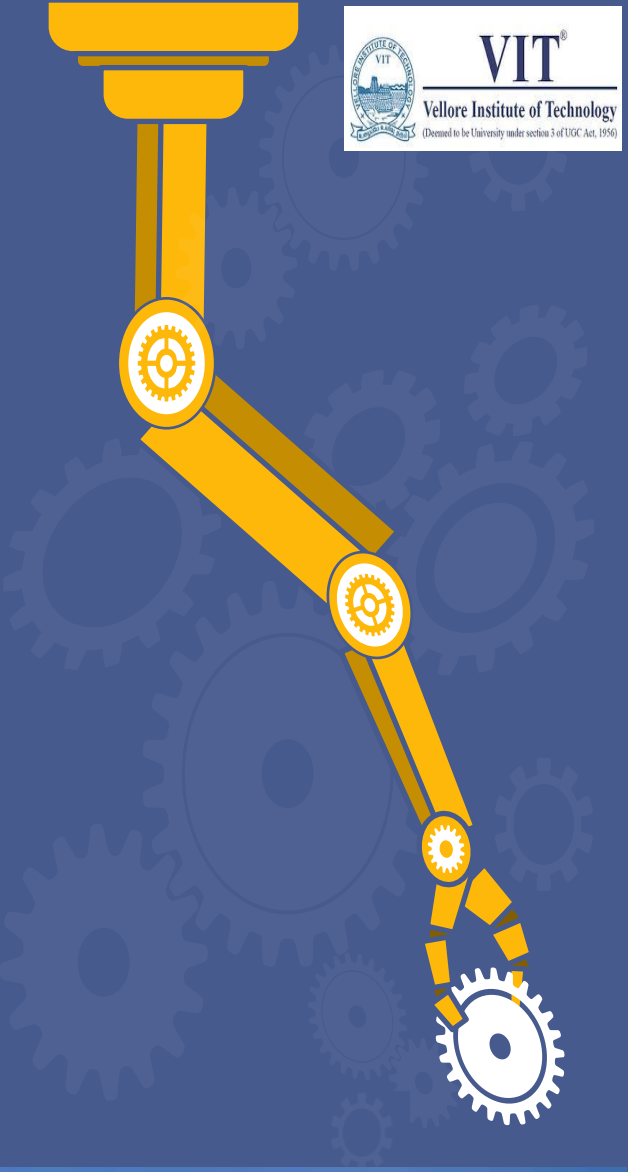
1. Apply the mathematical tools to analyse and derive the running time of the algorithms

2. Demonstrate the major algorithm design paradigms.

3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.

4. Articulating Randomized Algorithms.

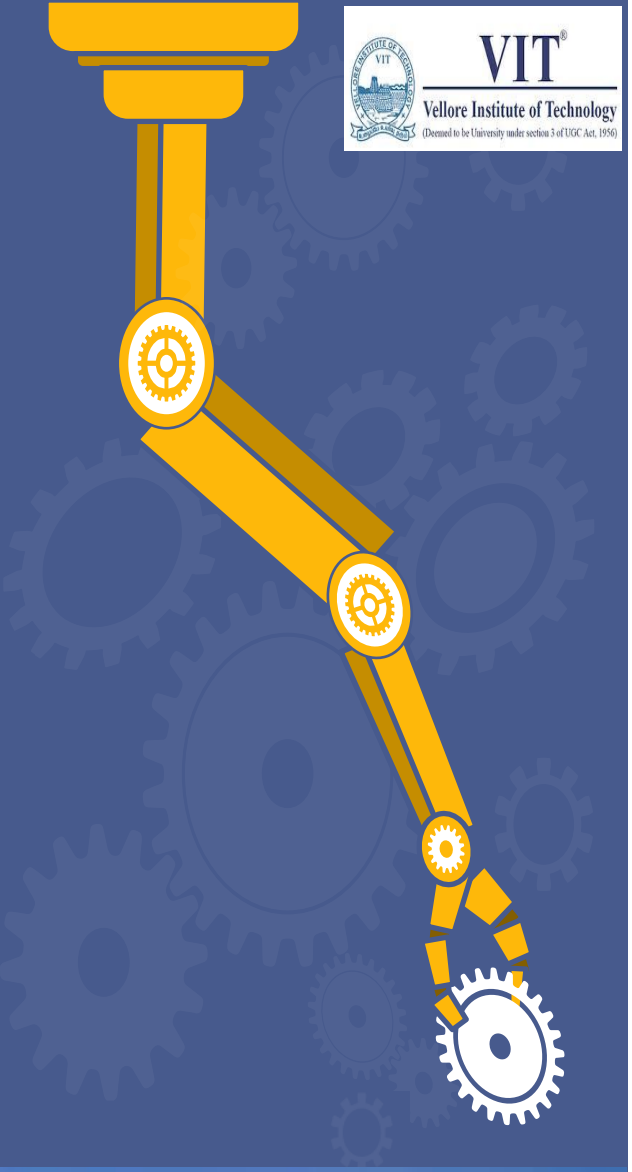
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	**Design Paradigms: Greedy, Divide and Conquer Techniques**	**6 hours**
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	**Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques**	**10 hours**
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	**String Matching Algorithms**	**5 hours**
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	**Graph Algorithms**	**6 hours**
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	**Geometric Algorithms**	**4 hours**
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	**Randomized algorithms**	**5 hours**
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	**Classes of Complexity and Approximation Algorithms**	**7 hours**
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	**Contemporary Issues**	**2 hours**



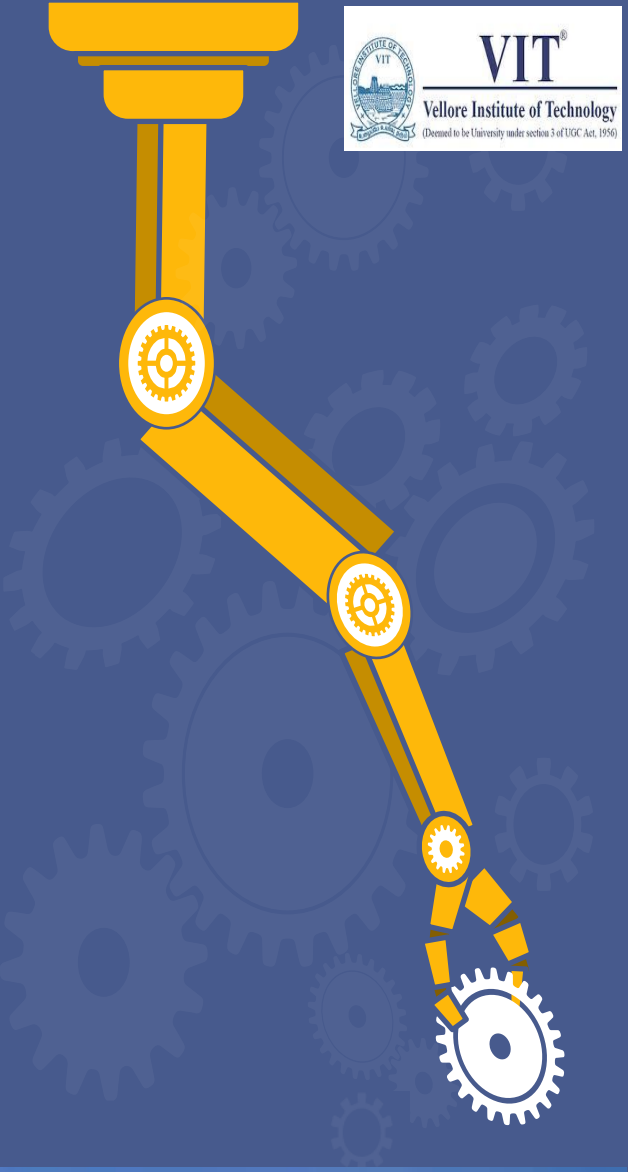
BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, ÉvaTardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press,1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B.Orlin, " Network Flows: Theory, Algorithms and Applications", Pearson Education, 2014.



Module 3: String Matching Algorithms(5 Hours)

- Naive String Matching Algorithm
- **KMP–Knuth Morris Pratt Algorithm**
- Rabin- Karp Algorithm
- Suffix Trees

String Matching

- Text-editing programs frequently need to find all occurrences of a pattern in the text.
- Typically, the text is a document being edited, and the pattern searched for is a particular word supplied by the user.
- Efficient algorithms for this problem called **"String Matching"**
- **Example:**
  - Search for particular patterns in DNA sequences
  - Internet search engines – find Web pages relevant to queries.

Knuth-Morris-Pratt Algorithm

- One of the most popular string matching algorithms used to find a Pattern in a Text
- IDEA : Compares character by character from left to right. But whenever a mismatch occurs, it uses a pre-processed table called **"Prefix Table"** to skip characters comparison while matching.
- Some times prefix table is also known as **LPS Table**. Here LPS stands for **"Longest proper Prefix which is also Suffix"**.

Knuth-Morris-Pratt Algorithm

- Pattern:- abcdabc
- proper prefixes:- a,ab,abc,abcd,abcda,abcdab
- Proper suffixes:- c,bc,abc,dabc,cdabc,bcdabc

Knuth-Morris-Pratt Algorithm

- abcdabeabf
- abcdeabfabc
- aabcadaabe
- aaaabaacd
- abcdabca
- aabaabaaa

- Try out finding LPS table for the above patterns

Knuth-Morris-Pratt Algorithm

TEXT

A	B	C	E	A	B	C	D	A	B	E	A	B	C	D	A	B	C	D	A	B	D	E

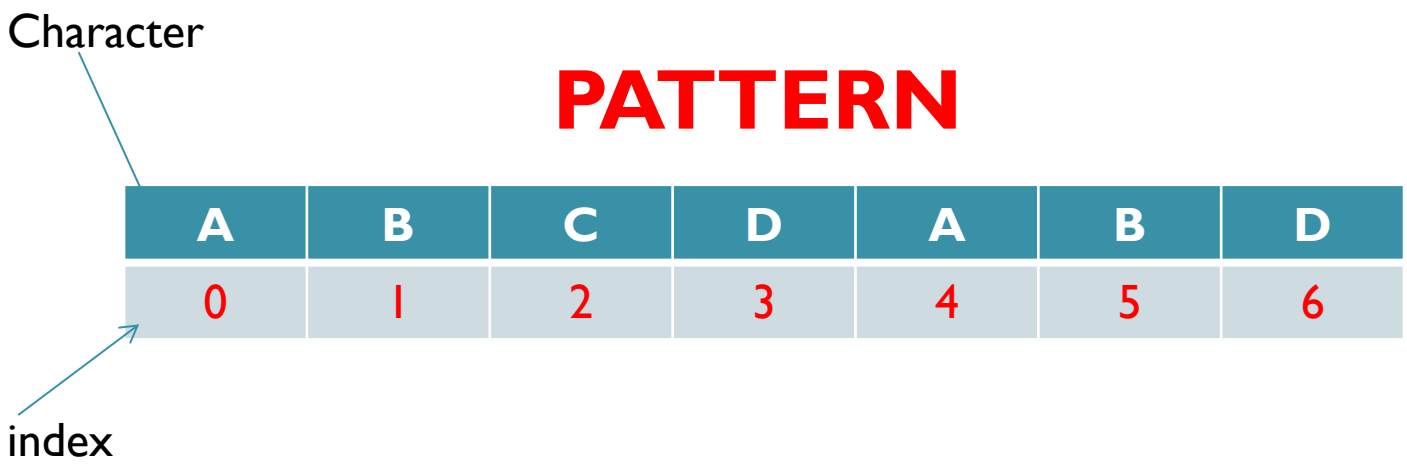
PATTERN

A	B	C	D	A	B	D

Steps for Creating LPS Table(Prefix Table)/π Table

- Step 1 - Define a one dimensional array with the size equal to the length of the Pattern. (LPS[size])
- Step 2 - Define variables i & j. Set i = 0, j = 1 and LPS[0] = 0.
- Step 3 - Compare characters at Pattern[i] and Pattern[j].
- Step 4 - If both are matched then set LPS[j] = i+1 and increment both i & j values by one. Goto to Step 3.
- Step 5 - If both are not matched then check the value of variable 'i'. **If '0'** then set LPS[j] = 0 and increment 'j' value by one, **if not '0'** then set i = LPS[i-1]. Goto Step 3.
- Step 6- Repeat above steps until all the values of LPS[] are filled.

Creating LPS Table(Prefix Table) for Given Pattern



Define LPS table of size 7 which is equal to length of the pattern

LPS

0	1	2	3	4	5	6

Creating LPS Table(Prefix Table) for Given Pattern

STEP-1: Define Variables i and j, **set i=0 & j=1** and LPS[0]=0

LPS

0	1	2	3	4	5	6
0						

i=0
J=1

STEP-2: Compare pattern[i] with pattern [j]
pattern[0] with pattern [1] // **i=0 & j=1**

A with B

Both are not matching and also i=0, so set LPS[j]=0 and j=j+1

LPS

0	1	2	3	4	5	6
0	0					

i=0
J=2

Creating LPS Table(Prefix Table) for Given Pattern

STEP-3: Compare pattern[i] with pattern [j]

pattern[0] with pattern [2] // **i=0 & j=2**

A with C

Both are not matching and also i=0, so set LPS[j]=0 and j=j+1

LPS

0	1	2	3	4	5	6
0	0	0				

i=0

J=3

Creating LPS Table(Prefix Table) for Given Pattern

STEP-4: Compare pattern[i] with pattern [j]

pattern[0] with pattern [3] // **i=0 & j=3**

A with D

Both are not matching and also i=0, so set LPS[j]=0 and j=j+1

LPS

0	1	2	3	4	5	6
0	0	0	0			

i=0

J=4

Creating LPS Table(Prefix Table) for Given Pattern

STEP-5: Compare pattern[i] with pattern [j]

pattern[0] with pattern [4] // **i=0 & j=4**

A with A

Both are matching, so set LPS[j]=i+1 and increment i and j by one.

LPS

0	1	2	3	4	5	6
0	0	0	0	1		

i=1

J=5

Creating LPS Table(Prefix Table) for Given Pattern

STEP-6: Compare pattern[i] with pattern [j]

pattern[1] with pattern [5] // **i=1 & j=5**

B with B

Both are matching, so set LPS[j]=i+1 and increment i and j by one.

LPS

0	1	2	3	4	5	6
0	0	0	0	1	2	

i=2

J=6

Creating LPS Table(Prefix Table) for Given Pattern

STEP-7: Compare pattern[i] with pattern [j]

pattern[2] with pattern [6] // **i=2 & j=6**

C with D

Both are NOT matching, and also i ! =0 , so set i= LPS[i-1].

LPS

0	1	2	3	4	5	6
0	0	0	0	1	2	

i=0

J=6

Creating LPS Table(Prefix Table) for Given Pattern

STEP-8: Compare pattern[i] with pattern [j]

pattern[0] with pattern [6] // **i=0 & j=6**

A with D

Both are NOT matching, and also i =0 , so set LPS[j]=0 and j=j+1

LPS

0	1	2	3	4	5	6
0	0	0	0	1	2	0

i=0

J=7

Once Table is filled, Stop the Process

Knuth-Morris-Pratt Algorithm

LPS Function or PREFIX Function(P)

$m = P.length$
let $\pi[1..m]$ be a new array
$\pi[1] = 0$
$k = 0$
for $q = 2$ **to** m
    while $k > 0$ and $P[k+1] \neq P[q]$
        $k = \pi[k]$
    if $P[k+1] == P[q]$
        $k = k + 1$
    $\pi[q] = k$
return π

LPS Function or PREFIX Function(P)

Step 1: q = 2, k = 0

⊓[2]= 0

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
⊓	0	0					

Step 2: q = 3, k = 0

⊓[3]= 1

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
⊓	0	0	1				

COMPUTE-PREFIX-FUNCTION(P)

```
1  m = P.length
2  let π[1..m] be a new array
3  π[1] = 0
4  k = 0
5  for q = 2 to m
6      while k > 0 and P[k + 1] ≠ P[q]
7          k = π[k]
8      if P[k + 1] == P[q]
9          k = k + 1
10     π[q] = k
11 return π
```

LPS Function or PREFIX Function(P)

Step 3: q = 4, k = 1

⊓[4]= 2

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
⊓	0	0	1	2			

Step 4: q = 5, k = 2

⊓[5]= 3

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
⊓	0	0	1	2	3		

COMPUTE-PREFIX-FUNCTION(P)

```
1  m = P.length
2  let π[1..m] be a new array
3  π[1] = 0
4  k = 0
5  for q = 2 to m
6      while k > 0 and P[k + 1] ≠ P[q]
7          k = π[k]
8      if P[k + 1] == P[q]
9          k = k + 1
10     π[q] = k
11 return π
```

LPS Function or PREFIX Function(P)

Step 5: q = 6, k = 3

⊓[6]= 1

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
⊓	0	0	1	2	3	0	

Step 6: q = 7, k = 1

⊓[7]= 1

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
⊓	0	0	1	2	3	0	1

COMPUTE-PREFIX-FUNCTION(P)

```
1  m = P.length
2  let π[1..m] be a new array
3  π[1] = 0
4  k = 0
5  for q = 2 to m
6      while k > 0 and P[k + 1] ≠ P[q]
7          k = π[k]
8      if P[k + 1] == P[q]
9          k = k + 1
10     π[q] = k
11 return π
```

How to use LPS Table(Prefix Table)

- LPS table is used to decide how many characters are to be skipped for comparison when a mismatch has occurred.
- When a mismatch occurs, check the LPS value of the previous character of the mismatched character in the pattern.
- If it is '0' then start comparing the **first character of the pattern** with the **next character to the mismatched character in the text.**
- If it is not '0' then start comparing the **character which is at an index value equal to the LPS value of the previous character** to the **mismatched character in pattern with the mismatched character in the Text.**

Knuth-Morris-Pratt Algorithm- Example

TEXT

A	B	C	E	A	B	C	D	A	B	E	A	B	C	D	A	B	C	D	A	B	D	E

PATTERN

A	B	C	D	A	B	D

LPS TABLE

0	1	2	3	4	5	6
0	0	0	0	1	2	0

Knuth-Morris-Pratt Algorithm- Step 1

Start Comparing first character of pattern with first character of text from left to right

A	B	C	E	A	B	C	D	A	B	E	A	B	C	D	A	B	C	D	A	B	D	E

A	B	C	D	A	B	D

Mismatch occurred at pattern[3], so check LPS[2], it is 0

If it is 0, start comparing first character of pattern with next character to the mismatched character in the text.

0	1	2	3	4	5	6
0	0	0	0	1	2	0

LPS TABLE

Knuth-Morris-Pratt Algorithm- Step 2

Start Comparing first character of pattern with next character to the mismatched character in the text

Mismatch occurred at pattern[6], so check LPS[5], it is 2

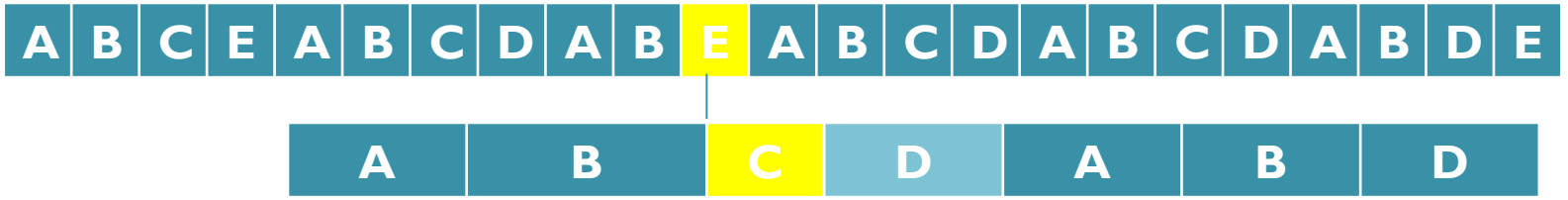
If it is not 0, start comparing pattern[2] character with mismatched character in the text.

0	1	2	3	4	5	6
0	0	0	0	1	2	0

LPS TABLE

Knuth-Morris-Pratt Algorithm- Step 3

LPS value is 2, no need to compare pattern[0] and pattern[1]



Mismatch occurred at pattern[2], so check LPS[1], it is 0

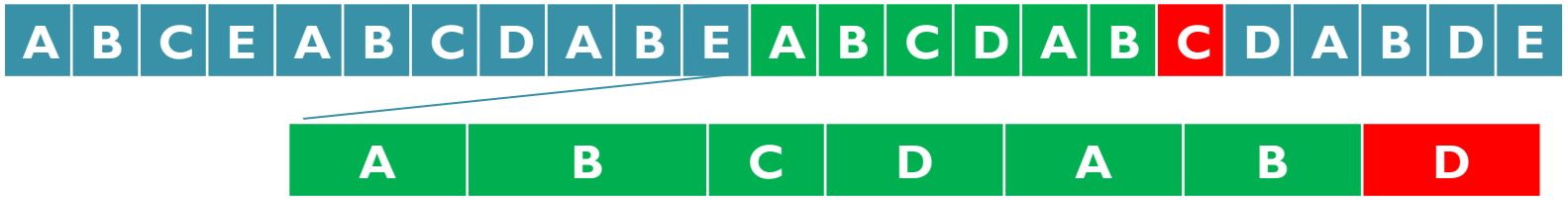
If it is 0, start comparing first character of pattern with next character to the mismatched character in the text.

0	1	2	3	4	5	6
0	0	0	0	1	2	0

LPS TABLE

Knuth-Morris-Pratt Algorithm- Step 4

Compare pattern[0] with next character in the text



Mismatch occurred at pattern[6], so check LPS[5], it is 2

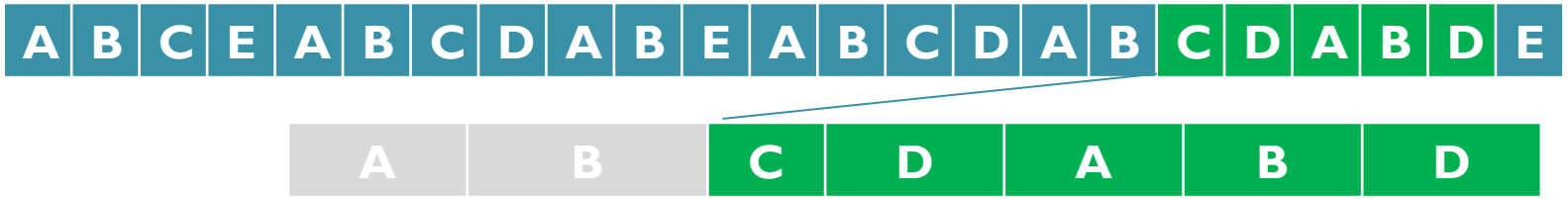
If it is not 0, start comparing pattern[2] character with mismatched character in the text.

0	1	2	3	4	5	6
0	0	0	0	1	2	0

LPS TABLE

Knuth-Morris-Pratt Algorithm- Step 5

Compare pattern[0] with next character in the text



Here all the characters of pattern matched with the substring of the text which is starting from index 15

0	1	2	3	4	5	6
0	0	0	0	1	2	0

LPS TABLE

Knuth-Morris-Pratt Algorithm

KMP Matcher(T,P)

```
n = T.length
m = P.length
π = COMPUTE-PREFIX-FUNCTION(P)
q = 0                                   // number of characters matched
for i = 1 to n                          // scan the text from left to right
    while q > 0 and P[q + 1] ≠ T[i]
        q = π[q]                        // next character does not match
    if P[q + 1] == T[i]
        q = q + 1                       // next character matches
    if q == m                           // is all of P matched?
        print "Pattern occurs with shift" i − m
        q = π[q]                        // look for the next match
```

Knuth-Morris-Pratt Algorithm

Time Analysis

- Preparing π table − O(m)
- Parsing through the text (main string) − O(n)
- Overall complexity − O(m + n)